浙江大学

计算机体系结构
实验报告

实验名称 ：	Lab1
姓名 ：	黄予恒
学号 ：	3240102750
学院/专业 ：	计算机学院 人工智能
实验地点 ：	东4-511
实验日期 ：	2026年 07 月 20 日
指导老师 ：	卜凯

1. Objectives and Requirements

Objectives:

- Finish the design of pipelined RV32I CPU
- Implement Pipeline Forwarding Detection and bypass unit
- Master the methods of 1-cycle stall and Predict-not-taken flush design

Requirements

- Understand RISC-V RV32I Instructions
- Understand how forwarding and stall works

2. Contents and Principles

First, there are four files we should complete this time:

- `CtrlUnit.v`
- `cmp_32.v`
- `RV32core.v`
- `HazardDetectionUnit.v`

Next I'll introduce the design of these components one by one.

For the `CtrlUnit.v`, we are asked to assign digital signal to certain wires.

```
...
wire BEQ = Bop && funct3_0;                          //to fill sth. in
wire BNE = Bop && funct3_1;                          //to fill sth. in
wire BLT = Bop && funct3_4;                          //to fill sth. in
...
```

These signals are for decoding the instruction, we only need to use the opcode and funct3 to implement the decoding.

Note that most of the instructions in the same type have the same opcode, but note that `LUI, AUIPC, JAL, JALR` all have **unique opcode**, so we can directly use the opcode to decode them.

For the signal `Branch`, I assign it `JAL | JALR | (B_valid && cmp_res)` to make sure that the branch signal is pulled up to one when unconditional jump or branch occurs.

For the `cmp_ctrl` signal, I extend the former decoded signal to work as **mask codes** to simplify the code implementation :

```
assign cmp_ctrl = {3{BEQ}} & cmp_EQ |
                  {3{BNE}} & cmp_NE | ...
```

Also note that the `cmp_EQ...` signals are set to the same as the implementation in `cmp_32.v`

Next are the ALUSrc control signal. The control signal here must be aligned with our wiring in the `RV32core.v`

Since for the first mux, I0 connects PC and I1 connects rs1_data, our ALUSrc_A should output 1 most of the time, only when `JALR, JAL, AUIPC` happens, should we connect PC to the ALU.

The design of ALUSrcB is the same.

Then for the `rs1use` and `rs2use` signals,

Not Finished This Part in CtrlUnit.v

For the `cmp_32.v` part, we only need to design a simple comparator.

we simply assign the output signal c as

`assign c = ((res_EQ && EQ)) || ...`, also res_XX as the potential result and XX as the mask code.

For the `RV32core.v` part, we connect all the wires

The `mux_IF` is for selecting the next PC for our PC register, so when the Branch_ctrl is pulled up, then we should feed the jump_PC back, so it should be connected to I1, and connect PC_4_IF to I0 as the default next PC.

The design of mux_forward should be consistent with the design in HazardDetectionUnit

Not Finished

For the `HazardDetectionUnit` part, Not Finished

3. Experiment Process

I will introduce how I debug in the whole process.

4. Analysis and Results

5. Discussion and Conclusions